

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Unknown State Transition Functions

Multi Object Tracking

Nov 20, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **In the last lectures we**
  - looked at the basic variant of the Kalman Filter,
  - interrelated it to Bayesian estimators,
  - and studied Extended and Unscented Kalman Fiters.
- **Today, we will**
  - do a quick excursion int the idea of unknown state transition functions, and
  - start to look at Multi Object Tracking applications.

Lecture Outline (bit late, I know)

- **Complementary Filter**
- **Machine Learning (basics, AdaBoost, DNNs, object detection with cameras)**
- **Stochastics (probability distributions, moments, covariance, covariance geometry)**
- **Kalman Filter (basic, Extended, Unscented)**
- **Multi Object Tracking**
  - Basics
  - Track Splitting
  - Multi-Hypothesis Tracking
  - Network Flow Optimization
  - Tracking DNNs

You are here

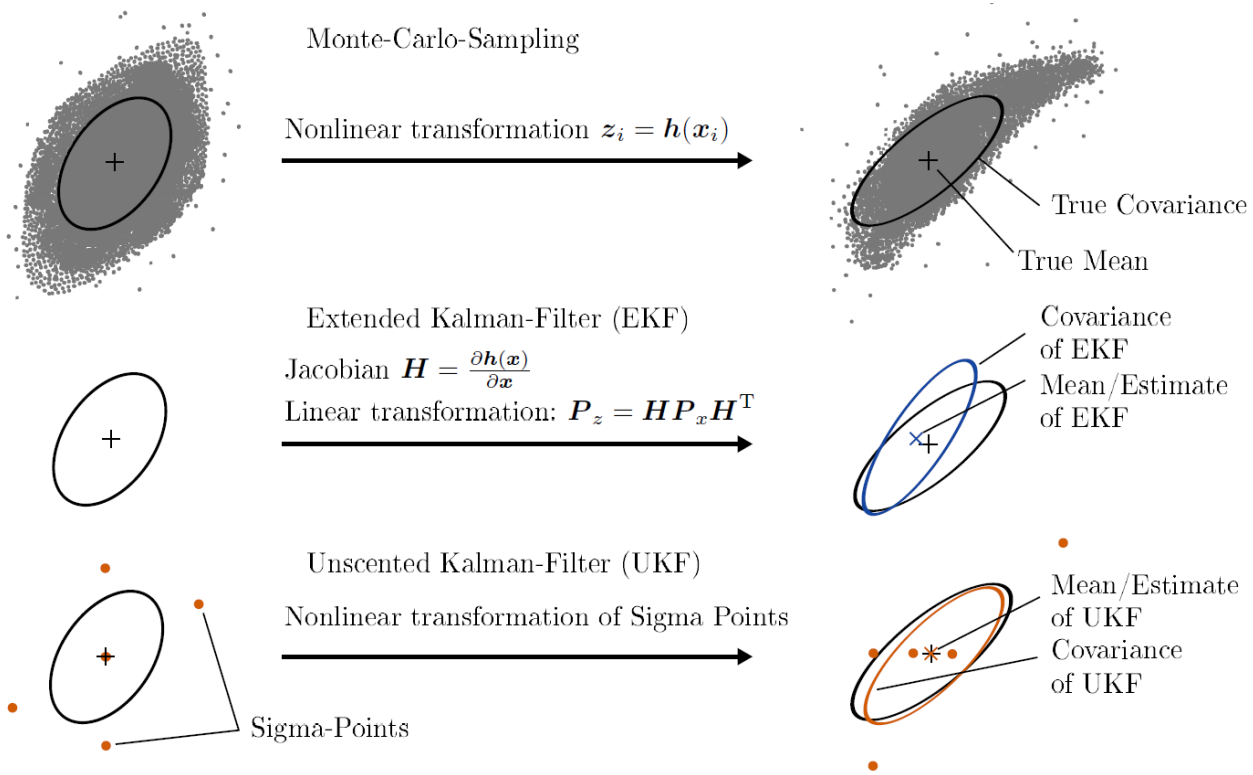
- **Particle Filters (basics, Joint Probability Data Association, applications)**
- **Simultaneous Localization and Mapping (Localization, map creation, loop closing)**
- **DNNs for sensor data fusion (Early Fusion &c.)**

Exercise 4 Solutions

Variants of the Kalman Filter

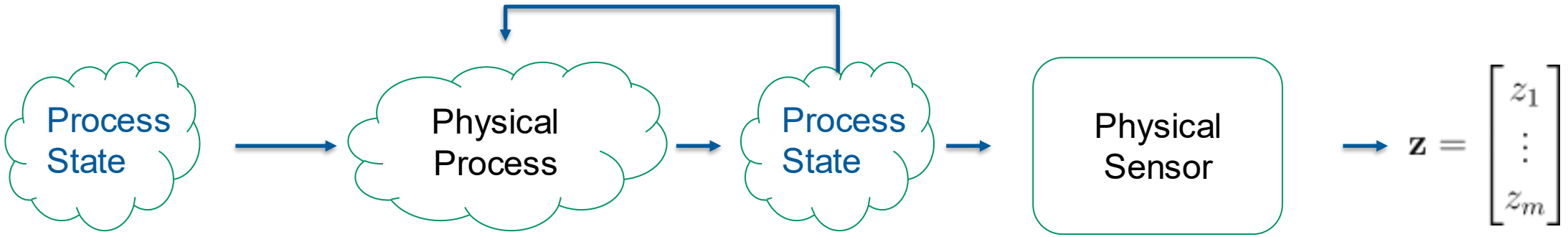
How to deal with State Transition Functions

Recap

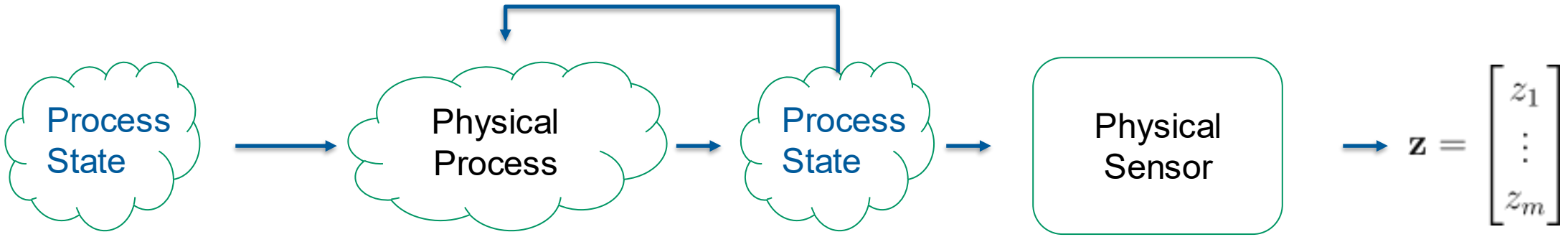


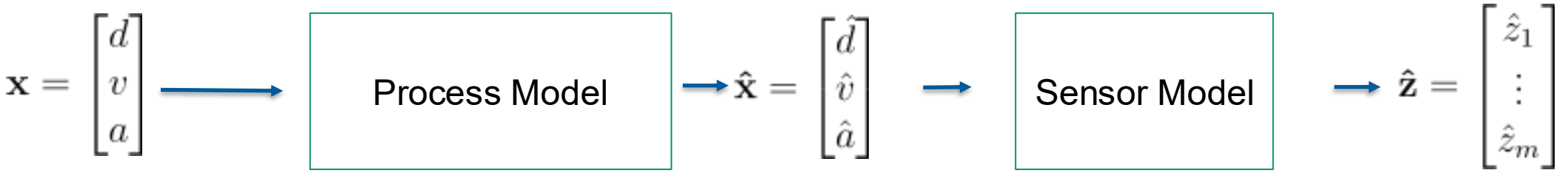
- **Even for UKF, we need to be able to at least sample the state transition function (even if we can't formulate it directly).**
- **What do we do if we don't have it available?**

Approximating State Transition Functions



Approximating State Transition Functions

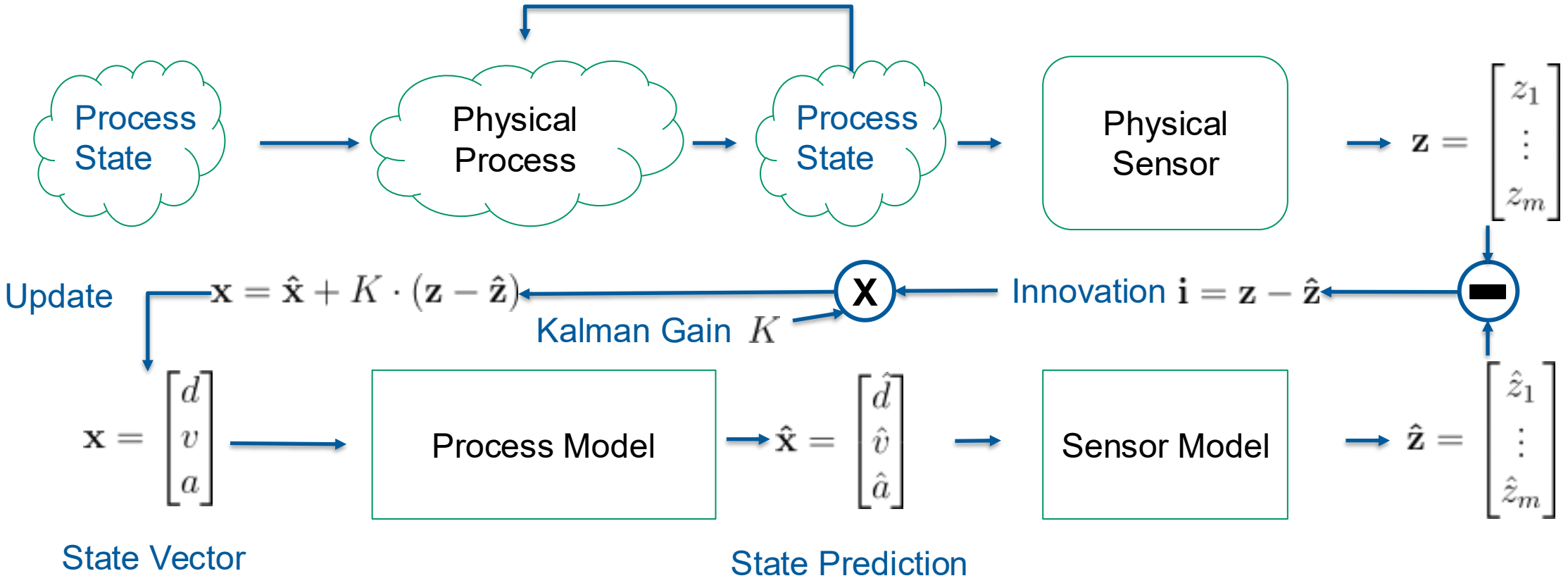




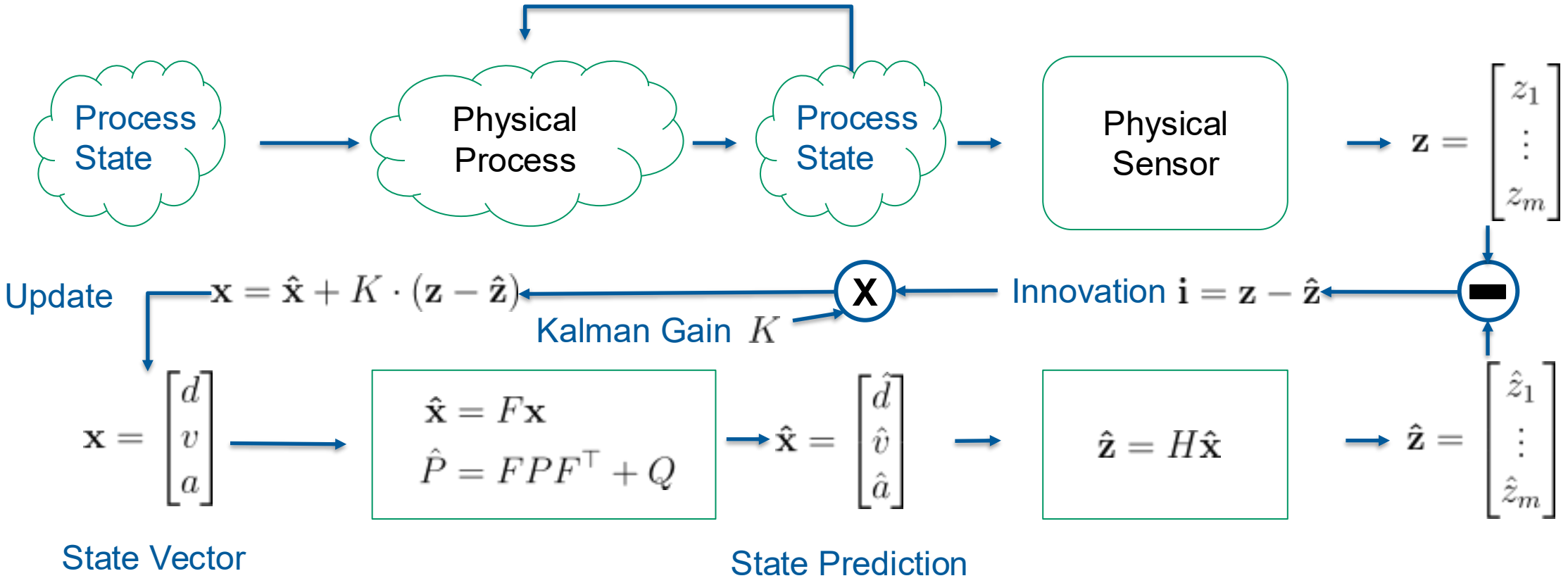
State Vector

State Prediction

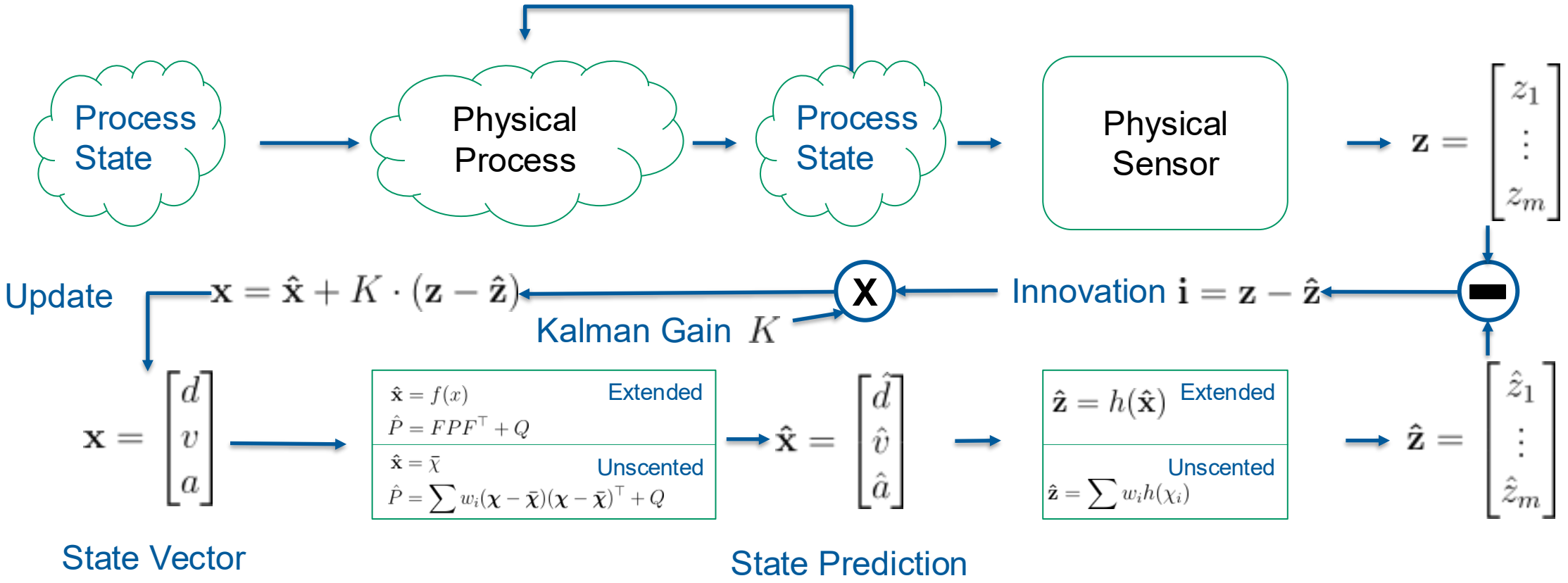
Approximating State Transition Functions



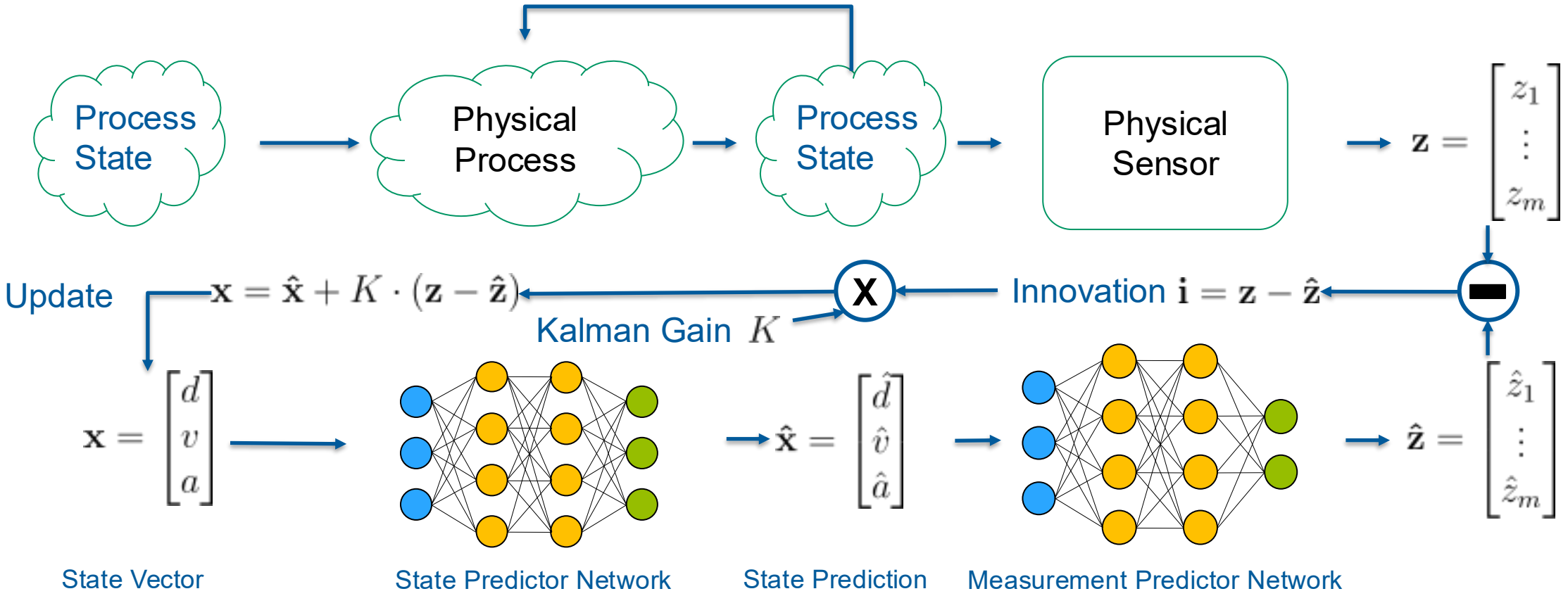
Approximating State Transition Functions



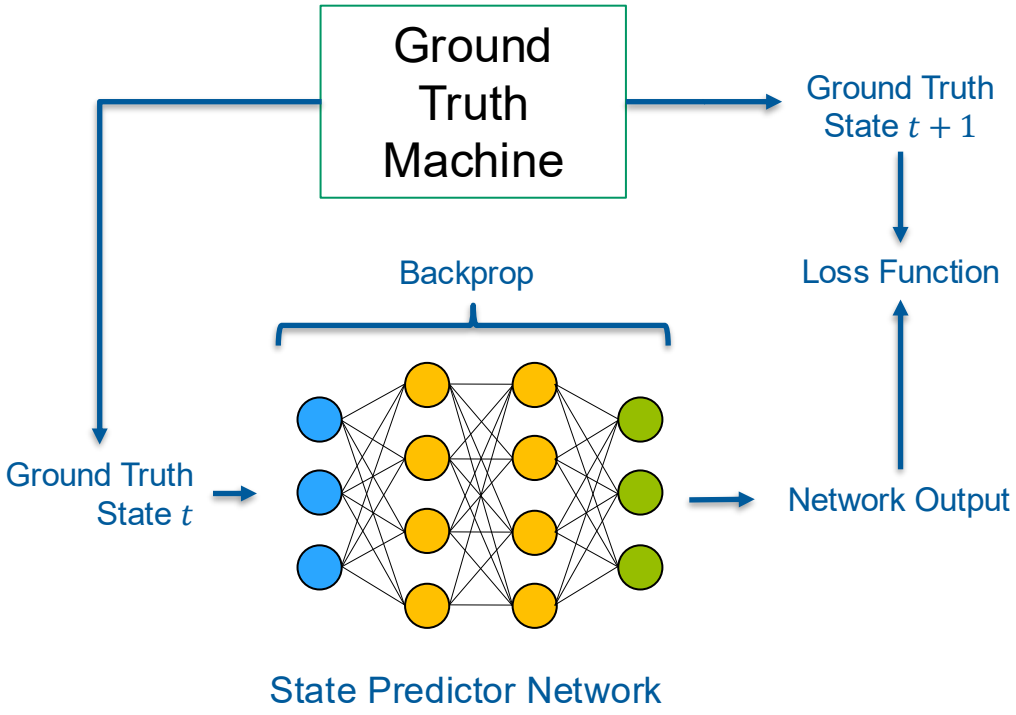
Approximating State Transition Functions

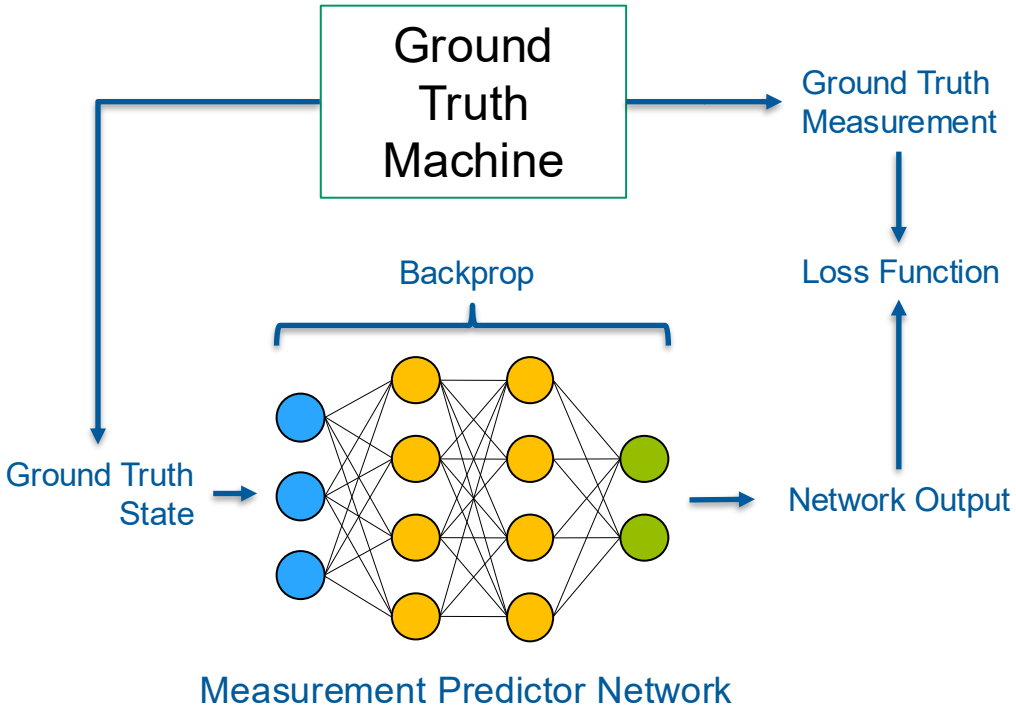


Approximating State Transition Functions



Training State and Measurement Predictor Networks





The Ground Truth Machine can be a simulator or a physical machine spiked with reference sensors.

Ground Truth Machines in the Wild



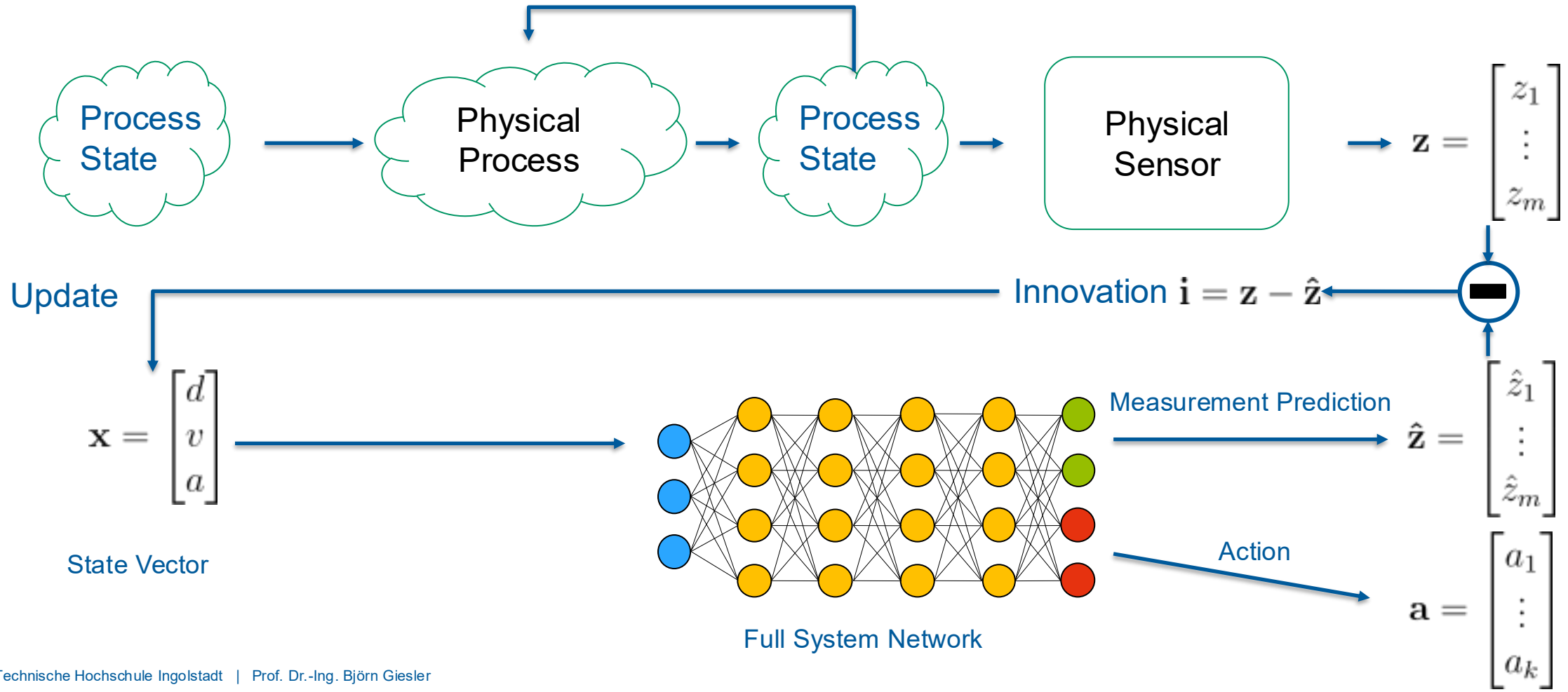








Other options



Full System Networks / End-To-End Autonomy

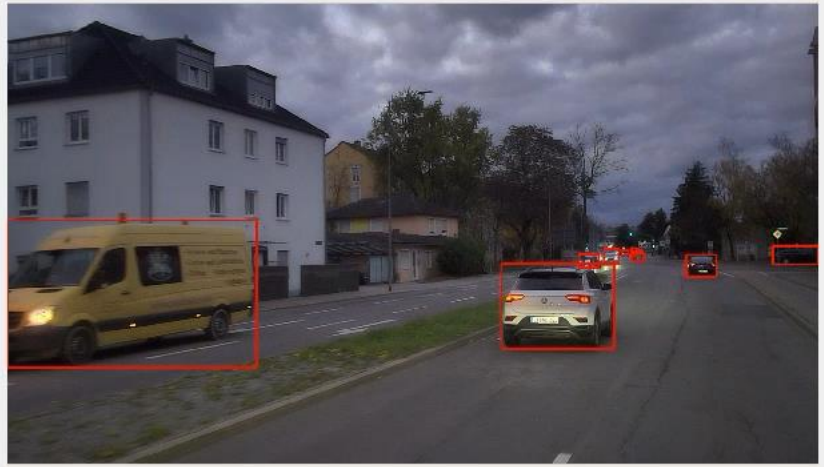
- **Full System Networks or End-To-End Networks are increasingly used in Autonomous Driving and Robotics.**
  - Must be stateful networks because state is no longer kept outside of the network.
    Time-Delay NNs, Recurrent NNs, LSTMs, Transformers or variants thereof.
  - Advantage: Can model the full system state without going through human-chosen abstractions.
  - Disadvantage: Hard to certify, hard to introspect, hard to understand, no explicit modeling of uncertainties.
  - Solution: Couple with classical systems that carry the safety load.
- **Very complex system behavior can only be modeled using end-to-end networks today.**
- **A prime example is humanoid or other legged robots that "look good while moving" (as opposed to simply work well).**
- **Training Input can be obtained by**
  - Motion Capture (e.g. for humanoid robots that need to move like a human)
  - Reinforcement Learning (pre-training in simulation, followed by real-world training with physical system)

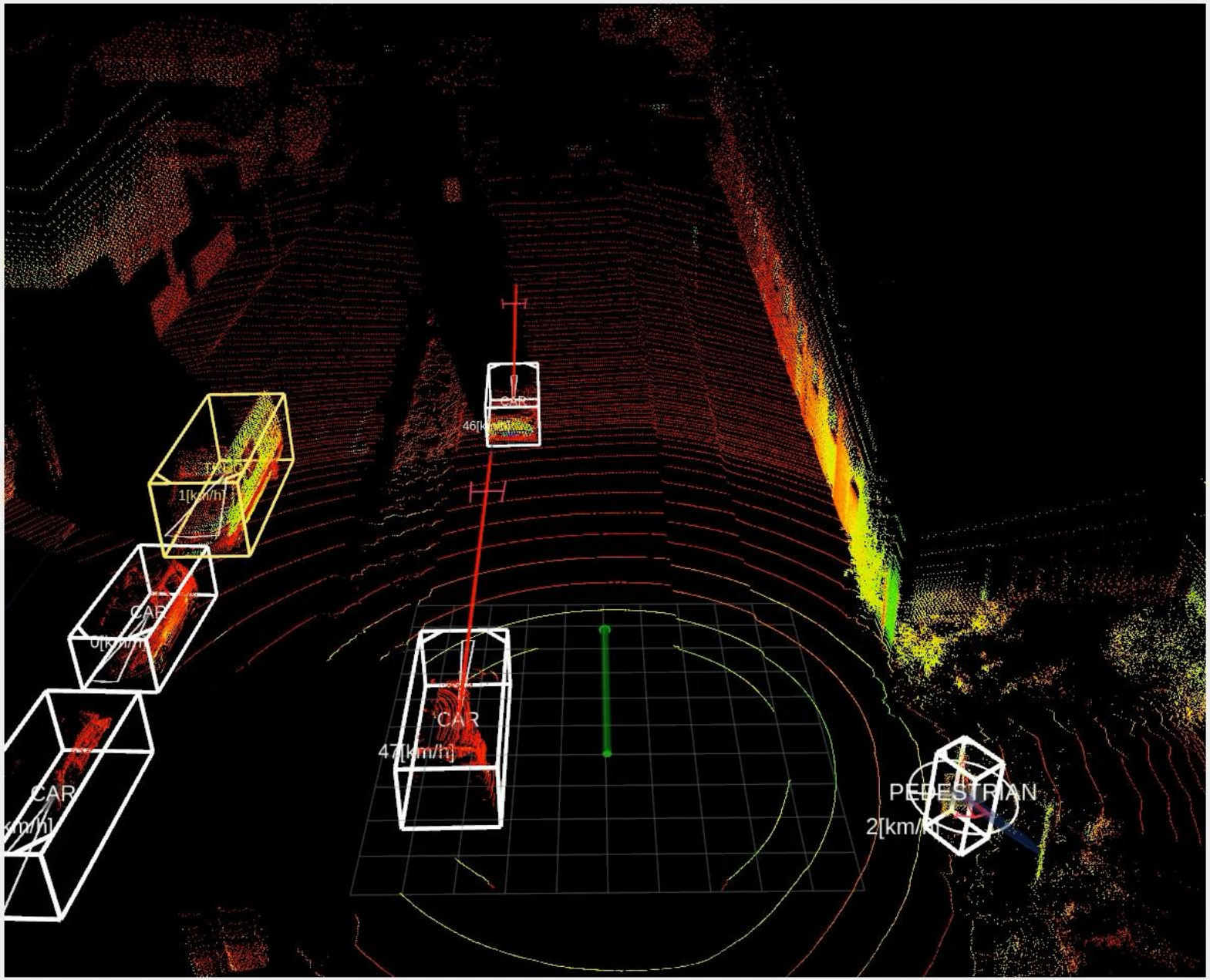
Multi Object Tracking (with Kalman Filters)

/home/audi/autoware.audiQ8/info/audiq8.rviz* - RViz

File Panels Help

Image

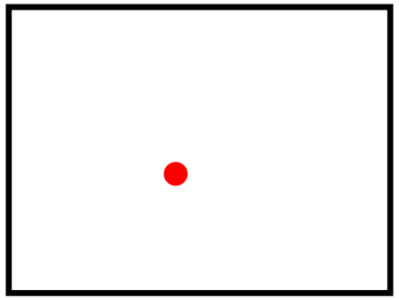




Reset

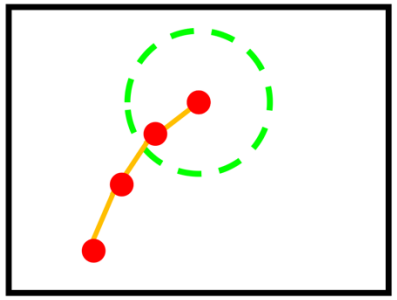
19 fps

Steps to Solve for Multi-Object Tracking



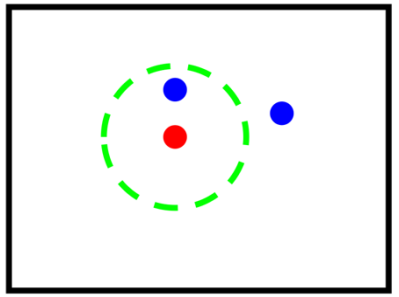
Detection

- We covered detection in the ML introduction, up to YOLO / YOLO3D.



Prediction / Innovation

- We covered this in the Kalman Filter lectures.
- We will continue to revisit this with different algorithms, as it's vitally important.



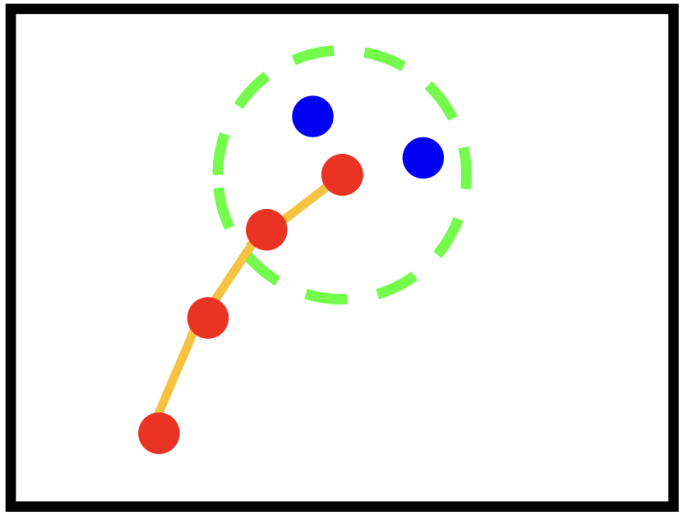
Data Association

- Association is the last unsolved step before we can close the loop.

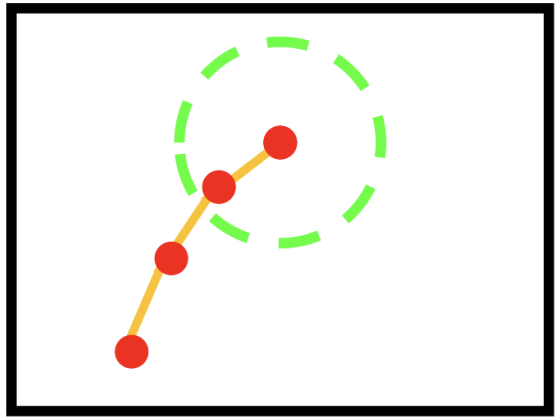
Motion Correspondence

- **Motion Correspondence Problem**
  - Do two measurements at the same time originate from the same object?

- **Why is it hard?**
  - First make predictions for the expected locations of the current set of objects
  - Match predictions to actual measurements
  - This is where ambiguities arise...



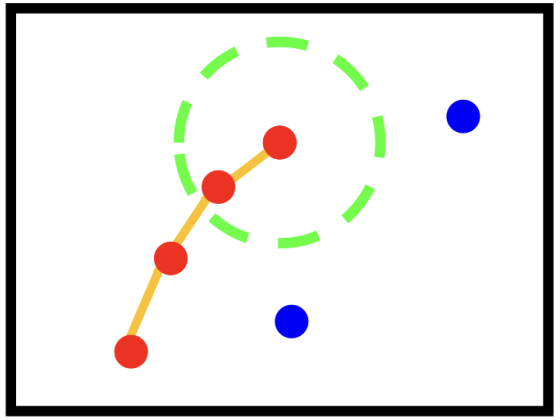
Motion Correspondence



1. Predictions may not be supported by measurements.

   Have the objects ceased to exist?
   ➔ remove them

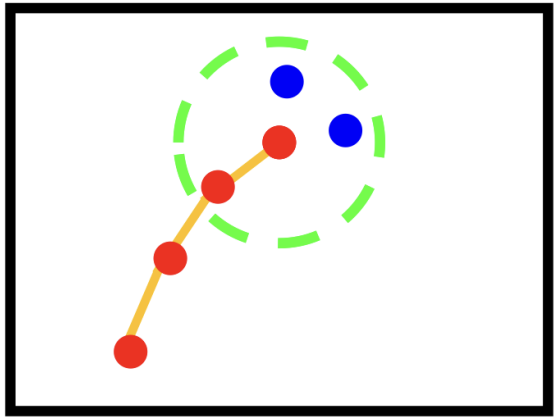
   Are they occluded?
   ➔ keep them



2. There may be unexpected measurements.

   Newly visible objects?
   ➔ set up tracks

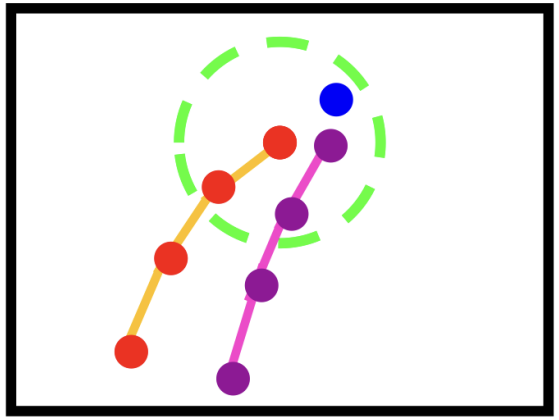
   Just noise?
   ➔ ignore them



3. More than one measurement may match a prediction.

   Which measurement is the correct one?

   What about the others?



4. A measurement may match to multiple predictions.

   Which track should the measurement be assigned to?

   What about the other track(s)?

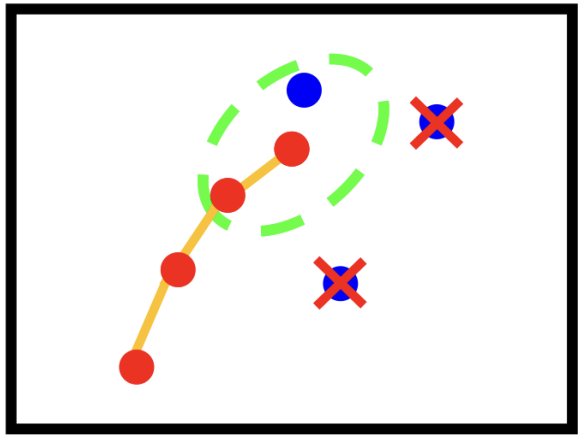
Multi-Object Tracking Problem

- We represent a track by a state vector, e.g. $\mathbf{x} = \left[x, y, v_x, v_y\right]^\top$
- As the track evolves, we denote its state by the time index t: $\mathbf{x}^{(t)} = [x^{(t)}, y^{(t)}, v_x^{(t)}, v_y^{(t)}]^\top$
- At each time step, we get a set of observations / measurements: $\mathbf{Y}^{(t)} = \{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$
- We now need to make the data association between tracks $\{\mathbf{x}_1^{(t)}, \ldots, \mathbf{x}_{N_t}^{(t)}\}$ and measurements $\{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$:
- Association $z_l^{(t)} = j$ iff $\mathbf{y}_j^{(t)}$ is associated with $\mathbf{x}_l^{(t)}$.

Simple Approach: Gating

- **Only consider measurements within a certain area around the predicted location.**
  ➔ Large gain in efficiency, since only a small region needs to be searched.
- **Nearest-Neighbor Filter**
  - Among the candidates in the gating region, only consider the one closest to the prediction $\hat{\mathbf{x}}$:

$$z_l^{(t)} = \underset{j}{\operatorname{argmin}} (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})^\top (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})$$

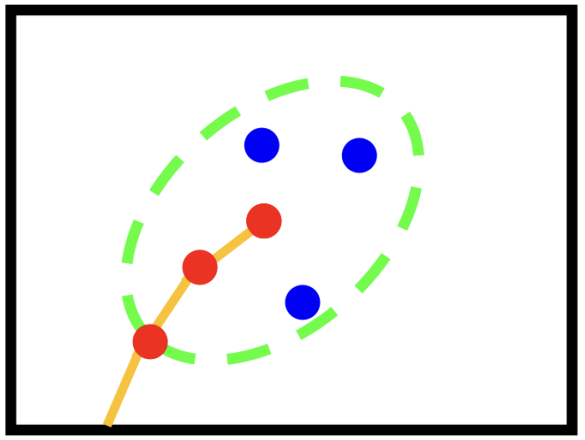


  - Better variant: Pick the candidate that is most likely under a Gaussian prediction model

$$z_l^{(t)} = \underset{j}{\operatorname{argmax}} \mathcal{N}(\mathbf{y}_j^{(t)}; \hat{\mathbf{x}}_l^{(t)}, P_l^{\prime(t)})$$

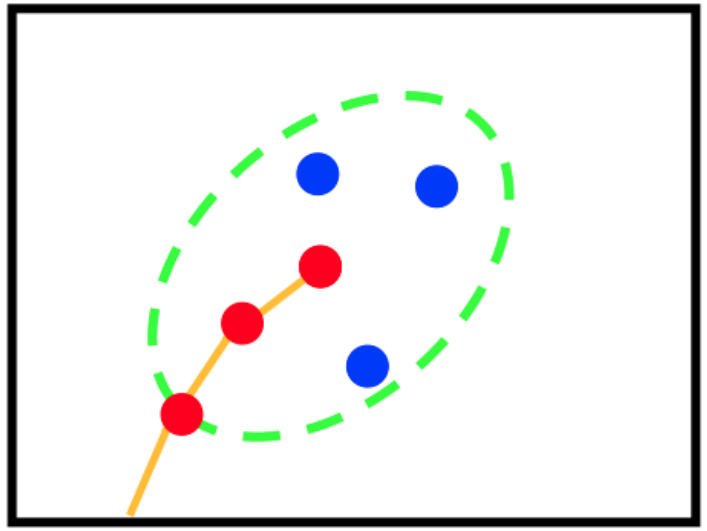
...which is commonly called the ***Mahalanobis Distance***:

$$z_l^{(t)} = \underset{j}{\operatorname{argmin}} (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})^\top P_l^{\prime -1} (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})$$



Gating with the Mahalanobis Distance

- **Provides exactly the quantities necessary to perform this**
  - Predicted mean location $\hat{\mathbf{x}}$
  - Prediction covariance P'
- **The Kalman filter prediction covariance also defines a useful gating area.**
  - E.g., choose the gating area size such that 95% of the probability mass is covered
- **Side note**
  - The Mahalanobis distance is *not* normal distributed, so can't simply follow the 3σ rule.
  - It follows a χ^2 distribution with the number of degrees of freedom equal to the dimension of x.
  - For a given probability bound, the corresponding threshold on the Mahalanobis distance can be got from χ^2 distribution tables.



Some Additional Notation

- Our Kalman Filter state of track $\mathbf{x}_l$ is given by the prediction and covariance $\hat{\mathbf{x}}_l^{(t)}, P_l'(t).$
- We have defined the *innovation* that measurement $\mathbf{z}_j$ brings to track $\mathbf{x}_l$ at time t as $\mathbf{i}_{j,l}^{(t)} = (\mathbf{z}_j^{(t)} - \hat{\mathbf{x}}_l^{(t)}).$
- With this, we can write the observation likelihood shortly as $$p(\mathbf{z}_j^{(t)}|\mathbf{x}_l^{(t)} \sim \exp\{-\frac{1}{2}\mathbf{i}_{j,l}^{(t)\top} P_l'^{-1}\mathbf{i}_{j,l}^{\top}\}.$$
- We can define the ellipsoidal gating or validation volume as $V^{(t)}(\gamma) = \{\mathbf{z} : (\mathbf{y} - \hat{\mathbf{x}}_l^{(t)})^\top P_l'^{(t)-1}(\mathbf{y} - \hat{\mathbf{x}}_l^{(t)}) \leq \gamma\}.$
- This is equivalent to saying "all $\mathbf{y}$ with a Mahalanobis distance $\leq \gamma$ ".

$$z_l^{(t)} = \underset{j}{\operatorname{argmin}}\ (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})^\top P_l'^{-1}(\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})$$

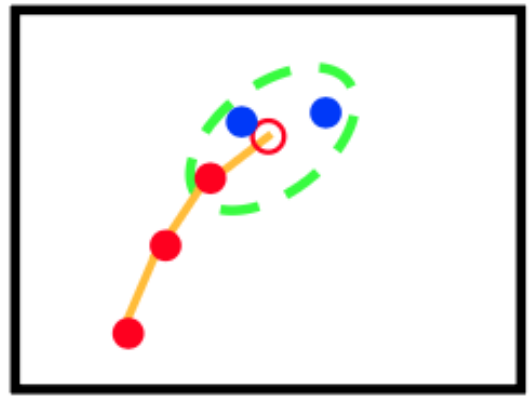
Limitation

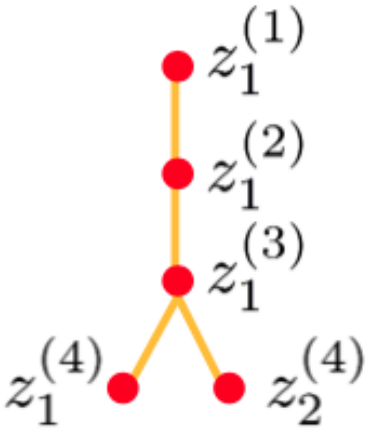
- **For NN assignments, there is always a finite chance that the association is incorrect, which can lead to serious effects.**
  - If a Kalman filter is used, a misassigned measurement may lead the filter to lose track of its target.
  - The NN filter makes assignment decisions only based on the current frame.
  - More information is available by examining subsequent sensor input..
  - Let’s make use of this information by postponing the decision process until a future frame will resolve the ambiguity...

Track Splitting Filter

Track Splitting Filter

- **The Nearest-Neighbor filter still has uncertainties and options that can't really be resolved**
  - Uncertain decision making
  - ...through gating decision making functions
- **Rather than arbitrarily assigning the "closest" (by whatever norm) measurement, form a tree.**
- **Branches denote alternate assignments. No *decision* is made at this stage; decisions are postponed until additional measurements have been gathered.**
- **Potential problem:**
  Track trees can quickly become very large due to combinatorial explosion.
- **We need some measure of the likelihood so we can prune the tree.**





Track Splitting Filter

- **Track likelihoods:**
  - Given a track l , let $\theta_{t,l}$ be the event that the sequence of assignments

$$Z_{t,l} = \{z^{(1)}_{i_1,l}, \ldots, z^{(t)}_{i_t,l}\}$$

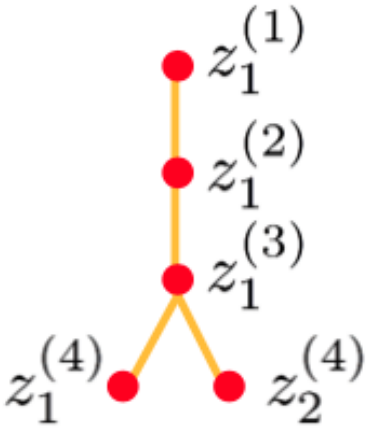
    from time 1 to t originate from the same object.

  - The likelihood of $\theta_{t,l}$ is the joint probability over all observations in the track:

$$L(\theta_{t,l}) = \prod_{j=1}^{t} p(z^{(j)}_{i_j,l} | Z_{j-1,l}, \theta_{k,l}).$$

  - If we assume Gaussian distributed observation likelihoods, this becomes

$$L(\theta_{t,l}) = \prod_{j=1}^{t} \frac{1}{\sqrt{2\pi\Sigma^{(j)}_l}} \exp\Big[-\frac{1}{2} \sum_{j=1}^{t} \mathbf{v}^{(j)\top}_{i_j,l} \Sigma^{(j)-1}_l \mathbf{v}^{(j)}_{i_j,l} \Big]$$



Track Spliting Filter

- **Starting from**

$$L(\theta_{t,l}) = \prod_{j=1}^{t} \frac{1}{\sqrt{2\pi\Sigma_l^{(j)}}} \exp\Big[-\frac{1}{2} \sum_{j=1}^{t} \mathbf{v}_{i_j,l}^{(j)\top} \Sigma_l^{(j)-1} \mathbf{v}_{i_j,l}^{(j)} \Big]$$

- **we use our old trick of simplifying with the log likelihood, eliminating the constant as well:**

$$\lambda_l(t) = -2 \log \left[L(\theta_{t,l}) \prod_{j=1}^{t} \sqrt{(2\pi\Sigma_i^{(j)}} \right]$$

$$= \sum_{j=1}^{t} \mathbf{v}_{i_j,l}^{(j)\top} \Sigma_l^{(j)-1} \mathbf{v}_{i_j,l}^{(j)}$$

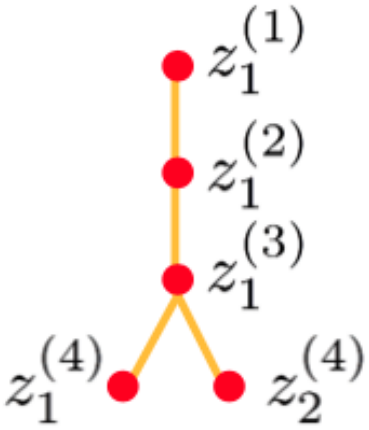
$$= \lambda_l(t-1) + \mathbf{v}_{i_j,l}^{(j)\top} \Sigma_l^{(j)-1} \mathbf{v}_{i_j,l}^{(j)}$$

Mahalanobis Distance

- **We can calculate this recursively as the sum of all Mahalanobis distances of all the measurements assigned to track l.**

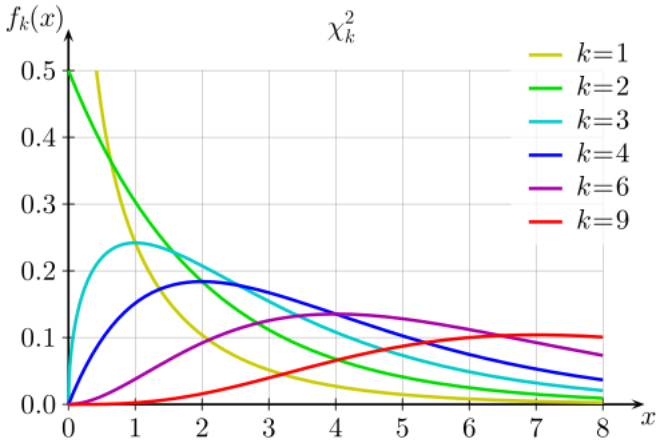
Track Splitting Filter

- **Instead of assigning the measurement that is *currently* closest (like in the nearest-neighbour algorithm),**
- **we can select the *sequence* of measurements that minimizes the *total* Mahalanobis distance over some interval.**
- **The modified log likelihood provides the merit of a particular node in the track tree.**
- **If we are working with Kalman filters, we have calculated most of these terms anyway.**
- **Problem:**
  - The track tree grows exponentially over time.
  - Need to generate a possibly very large number of tracks.



Track Splitting Filter – Pruning Strategies

- **In order to keep this feasible, need to apply pruning.**
- **Deleting unlikely tracks**
  - May be accomplished by comparing $\lambda(t)$, which follows a χ^2 distribution with $t \cdot n_z$ degrees of freedom, with a threshold α (set according to χ^2 tables).
  - In long tracks, $\lambda(t)$ gets dominated by old terms and responds very slowly to new ones.
  - ➔ Use sliding window to limit track length, or implement exponential decay.
- **Merging track nodes**
  - If the state estimates of two track nodes are similar, merge them.
  - E.g. if both tracks validate identical subsequent measurments.
- **Rank tracks based on $\lambda(t)$, and only keep the most likely N tracks.**



χ^2 distribution for different degrees of freedom

Track Splitting Filter

- **Summary:**
  - Very old algorithm (1975: P.Smith &al., *A Branching Algorithm for Discriminating and Tracking Multiple Objects*)
  - Considerable improvement over nearest-neighbour assignment.
  - Assignment decisions are delayed until more information is available.
- **Remaining problems**
  - Exponential complexity, requires heuristic pruning.
  - Merging of track nodes is necessary, because tracks may share measurements, which is physically unrealistic.

Outlook for the Next Lectures

- **More powerful approaches**
  - Multi-Hypothesis Tracking (MHT): Well-suited for KF, EKF approaches [Reid, 1979]
  - Joint Probabilistic Data Association Filters (JPDAF): Well-suited for PF approaches [Fortmann, 1983]
- **Data association as convex optimization problem**
  - Bipartite Graph Matching (Hungarian algorithm)
  - Network Flow Optimization
  - Efficient, globally optimal solutions for subclass of problems.

Literature / Links

1. **P.Smith &al., *A Branching Algorithm for Discriminating and Tracking Multiple Objects.* https://ieeexplore.ieee.org/document/1100851**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!